3.3 Padding and stride

- in the previous example of Figure 3, our input had both a height and width of 3 and our convolution kernel had both a height and width of 2, yielding an output representation with dimension 2×2
- as we generalized in Section 3.2, assuming that the input shape is $n_h \times n_w$ and the convolution kernel shape is $k_h \times k_w$, then the output shape will be $(n_h - k_h + 1) \times (n_w - k_w + 1)$
- therefore, the output shape of the convolutional layer is determined by the shape of the input and the shape of the convolution kernel

3.3 Padding and stride

- in several cases, we incorporate techniques, including *padding* and *strided convolutions*, that affect the size of the output
- as motivation, note that, since kernels generally have width and height greater than 1, after applying many successive convolutions, we tend to wind up with outputs that are considerably smaller than our input
- if we start with a 240×240 pixel image, 10 layers of 5×5 convolutions reduce the image to 200×200 pixels, slicing off 30% of the image, and with it ignoring any interesting information on the boundaries of the original image
- *padding* is the most popular tool for handling this issue
- in other cases, we may want to reduce the dimensionality drastically, e.g., if we find the original input resolution to be too big
- *strided convolutions* are a popular technique that can help in these instances

3.3 Padding and stride

- as described above, one tricky issue when applying convolutional layers is that we tend to lose pixels on the perimeter of our image
- since we typically use small kernels, for any given convolution, we might only lose a few pixels, but this can add up as we apply many successive convolutional layers
- one straightforward solution to this problem is to add extra pixels around the boundary of our input image, thus increasing the effective size of the image
- typically, we set the values of the extra pixels to zero
- in Figure 4, we pad a 3×3 input, increasing its size to 5×5; the corresponding output then increases to a 4×4 matrix
- the shaded portions are the first output element, as well as the input and kernel tensor elements used for the output computation: $0 \times 0 + 0 \times 1 + 0 \times 2 + 0 \times 3 = 0$

3.3 Padding and stride

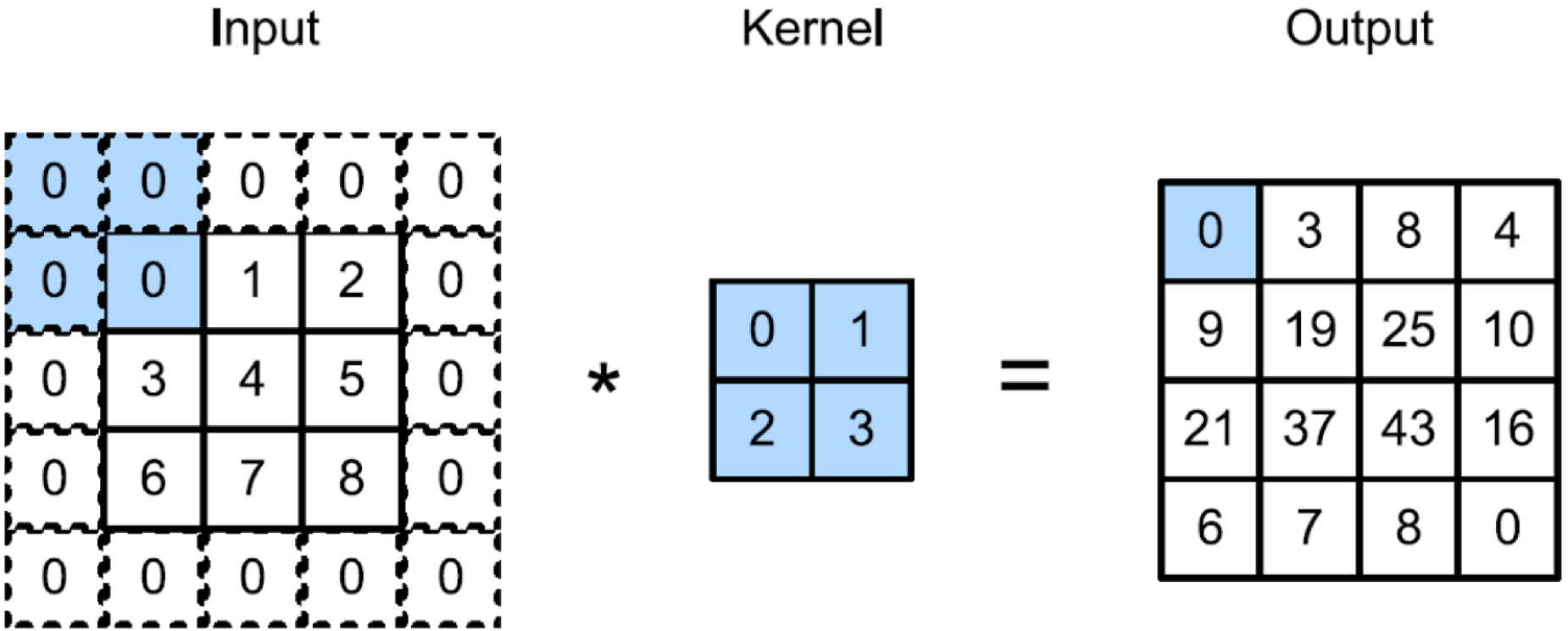

Figure 4: Two-dimensional cross-correlation with padding.

3.3 Padding and stride

- in general, if we add a total of p_h rows of padding (roughly half on top and half on bottom) and a total of p_w columns of padding (roughly half on the left and half on the right), the output shape will be:

$$(n_h - k_h + p_h + 1) \times (n_w - k_w + p_w + 1).$$

- this means that the height and width of the output will increase by p_h and p_w, respectively
- in many cases, we will want to set $p_h = k_h - 1$ and $p_w = k_w - 1$ to give the input and output the same height and width
- this will make it easier to predict the output shape of each layer, when constructing the network
- assuming that k_h is odd here, we will pad $p_h/2$ rows on both sides of the height
- if k_h is even, one possibility is to pad $\lceil p_h/2 \rceil$ rows on the top of the input and $\lfloor p_h/2 \rfloor$ rows on the bottom
- we will proceed in the same way for the width

3.3 Padding and stride

- CNNs commonly use convolution kernels with odd height and width values, such as 1, 3, 5, or 7
- choosing odd kernel sizes has the benefit that we can preserve the spatial dimensionality while padding with the same number of rows on top and bottom, and the same number of columns on left and right
- moreover, this practice of using odd kernels and padding to precisely preserve dimensionality offers a clear benefit
- for any two-dimensional tensor $\mathbf{X}$, when the kernel's size is odd and the number of padding rows and columns on all sides are the same, producing an output with the same height and width as the input, we know that the output $[\mathbf{Y}]_{i,j}$ is calculated by cross-correlation of the input and convolution kernel with the window centered on $[\mathbf{X}]_{i,j}$

3.3 Padding and stride

- when computing the cross-correlation, we start with the convolution window at the upper-left corner of the input tensor, and then slide it over all locations both down and to the right
- in previous examples, we default to sliding one element at a time
- however, sometimes, either for computational efficiency, or because we wish to downsample, we move our window more than one element at a time, skipping the intermediate locations
- we refer to the number of rows and columns traversed per slide as the *stride*
- so far, we have used strides of 1, both for height and width; sometimes, we may want to use a larger stride

3.3 Padding and stride

- Figure 5 shows a two-dimensional cross-correlation operation with a stride of 3 vertically and 2 horizontally
- the shaded portions are the output elements, as well as the input and kernel tensor elements used for the output computation: $0 \times 0 + 0 \times 1 + 1 \times 2 + 2 \times 3 = 8$, $0 \times 0 + 6 \times 1 + 0 \times 2 + 0 \times 3 = 6$
- we can see that, when the second element of the first column is outputted, the convolution window slides down three rows
- the convolution window slides two columns to the right when the second element of the first row is outputted
- when the convolution window continues to slide two columns to the right on the input, there is no output, because the input element cannot fill the window (unless we add another column of padding)

3.3 Padding and stride

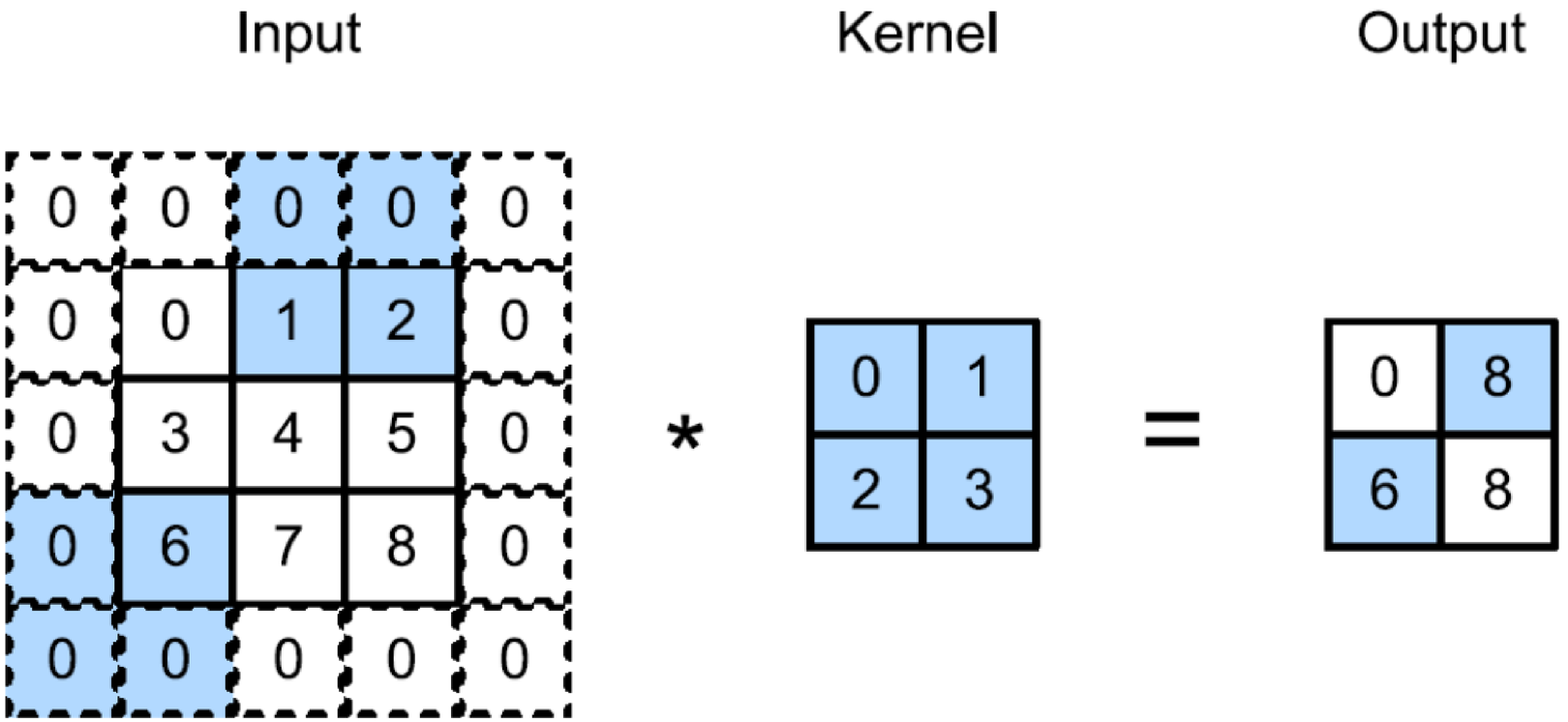

Figure 5: Cross-correlation with strides of 3 and 2 for height and width, respectively.

3.3 Padding and stride

- in general, when the stride for the height is s_h and the stride for the width is s_w, the output shape is:

$$\lfloor (n_h - k_h + p_h + s_h)/s_h \rfloor \times \lfloor (n_w - k_w + p_w + s_w)/s_w \rfloor.$$

- if we set $p_h = k_h - 1$ and $p_w = k_w - 1$, then the output shape will be simplified to $\lfloor (n_h + s_h - 1)/s_h \rfloor \times \lfloor (n_w + s_w - 1)/s_w \rfloor$
- going a step further, if the input height and width are divisible by the strides on the height and width, then the output shape will be $(n_h/s_h) \times (n_w/s_w)$

3.3 Padding and stride

- for the sake of brevity, when the padding number on both sides of the input height and width are p_h and p_w respectively, we call the padding (p_h, p_w)
- specifically, when $p_h = p_w = p$, the padding is p
- when the strides on the height and width are s_h and s_w, respectively, we call the stride (s_h, s_w)
- specifically, when $s_h = s_w = s$, the stride is s
- by default, the padding is 0 and the stride is 1
- in practice, we rarely use inhomogeneous strides or padding, i.e., we usually have $p_h = p_w$ and $s_h = s_w$